

**SIMATS ENGINEERING**  
**Co3-ASSESSMENT TOOL 2**  
**Case study**

**COURSE NAME:** Computer Networks

**COURSE CODE:** CSA07

---

**COURSE OUTCOME COVERED**

**CO3:** *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

**QUESTIONS:4**

**Total Marks:40**

---

***Code of Conduct:***

*I Harish Kumar / 192521340 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

**QUESTIONS:4**

**Total Marks:40**

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss**, **180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

## Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

### Answer:

**1) Transport protocol vs. application layer — root-cause analysis:** The dominant cause here is the transport-layer protocol choice/behaviour, with the application-layer buffering strategy acting as a significant secondary/amplifying factor. Real-time video-conferencing media should be carried over UDP (typically via RTP/SRTP, or a modern QUIC-based transport), not TCP. TCP's reliability guarantees — retransmission of lost segments and strict in-order delivery — are precisely wrong for real-time media: if even one segment is lost, TCP must retransmit it and hold up every later segment behind it until that retransmission succeeds ("head-of-line blocking"), and its congestion-control algorithm additionally cuts the sending rate sharply on every loss event. Given the reported 8% packet loss, if the platform is using (or falling back to) TCP-style reliable delivery for its media stream, that single fact would directly explain all three symptoms: voice distortion and frozen video frames (waiting on retransmitted, now-stale data before anything more can be played) and fluctuating bandwidth (TCP's congestion window repeatedly backing off and re-growing — the classic "TCP sawtooth"). The 180 ms jitter compounds this: if the application layer's jitter buffer is a large, static buffer designed to smooth out network timing variation, it converts transient jitter into user-perceptible freezing while it waits to reassemble a fully ordered playout sequence, rather than degrading gracefully.

#### Effect of Packet Loss: TCP (media over TCP) vs. UDP+RTP

##### TCP



*Result: frozen video frame, growing delay, congestion-control rate cut*

##### UDP + RTP



*Result: brief glitch/voice distortion only — no cascading delay*

*TCP's reliability + in-order delivery (head-of-line blocking) actively harms real-time media;  
UDP-based transport degrades gracefully instead of freezing.*

Figure 1 – How the same packet loss event plays out very differently under TCP vs. UDP+RTP.

**2) Justification based on real-time multimedia requirements:** Real-time audio/video has a distinctive “loss-tolerant, delay-intolerant” requirement profile: humans can often mentally interpolate or simply tolerate a brief glitch, dropped frame, or moment of static far better than they can tolerate an accumulating delay that makes a live conversation feel unnatural (ITU-T G.114 recommends keeping one-way latency under roughly 150 ms for acceptable interactivity). This maps directly onto UDP's design: UDP provides no retransmission and no in-order-delivery guarantee, so a lost packet simply results in one missing/degraded frame rather than blocking everything that follows — exactly the trade-off real-time media wants. TCP, by contrast, was designed for the opposite priority: guaranteed, ordered, complete delivery (ideal for file transfer or a web page, where correctness matters more than a few milliseconds), which is why its retransmission and congestion-control mechanisms actively work against, rather than for, a live video call under lossy conditions.

### 3) Recommended protocol-level and application-level improvements:

- **Transport layer:** Ensure media streams use UDP-based RTP/SRTP (or a QUIC-based transport) rather than TCP; use lightweight Forward Error Correction (FEC) or selective/NACK-based retransmission with strict deadlines instead of full TCP-style reliability.
- **Congestion/rate adaptation:** Deploy real-time adaptive bitrate control (e.g. WebRTC's Google Congestion Control) driven by RTCP receiver reports, so video quality/resolution scales down smoothly under loss/jitter instead of the stream freezing.
- **Jitter buffer tuning:** Replace any large, fixed-size jitter buffer with a dynamically-sized adaptive buffer that tracks measured jitter in real time, absorbing the 180 ms jitter without adding more delay than necessary.
- **Network-level QoS:** Apply DiffServ/DSCP traffic marking on the corporate network to prioritize real-time conferencing traffic ahead of routine bulk data, reducing loss and jitter for the media stream specifically.

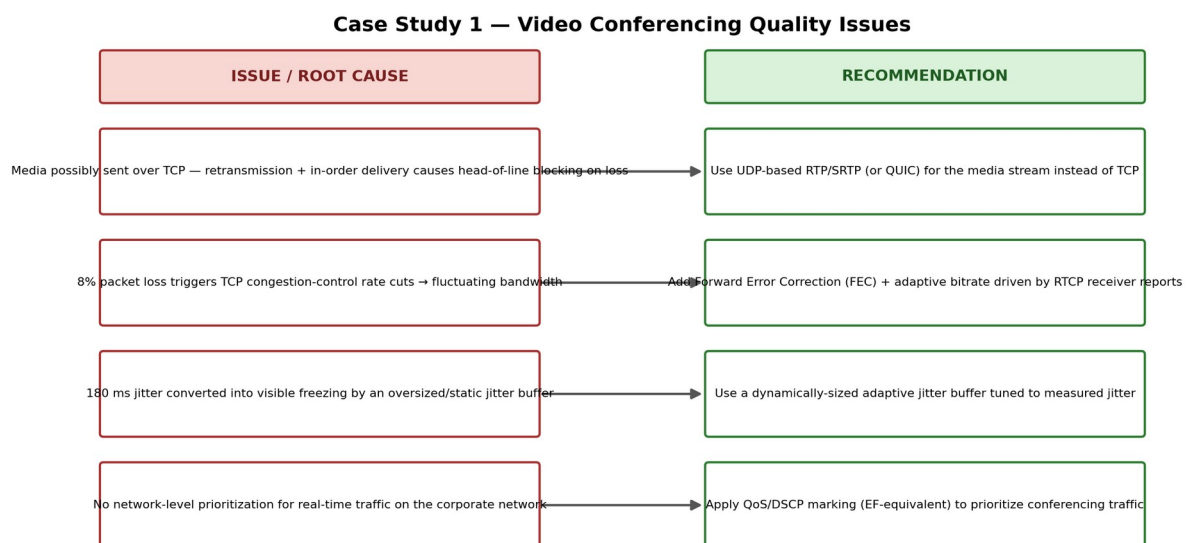


Figure 2 – Summary of Case Study 1 issues mapped to recommended fixes.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

### Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

### Answer:

#### 1) Reasons behind the high (~800 ms) DNS resolution time:

- **Geographic distance:** with a single (or centrally located) authoritative server, users on other continents incur substantial propagation delay purely from physical distance — intercontinental round trips alone can run into the hundreds of milliseconds.
- **Ineffective caching:** short or poorly-tuned TTLs mean resolvers cannot serve repeat queries from cache, forcing a full resolution round trip far more often than necessary.
- **Long resolution chains:** if recursive resolvers are frequently missing cache and must walk the hierarchy (root → TLD → authoritative) rather than answering locally, each additional hop adds its own round-trip delay.
- **Server-side load:** a single, non-Anycast authoritative server serving a global user base can itself become a processing/queuing bottleneck under load, adding further delay on top of network propagation time.

#### 2) Proposed DNS architecture to bring lookups below 50 ms:

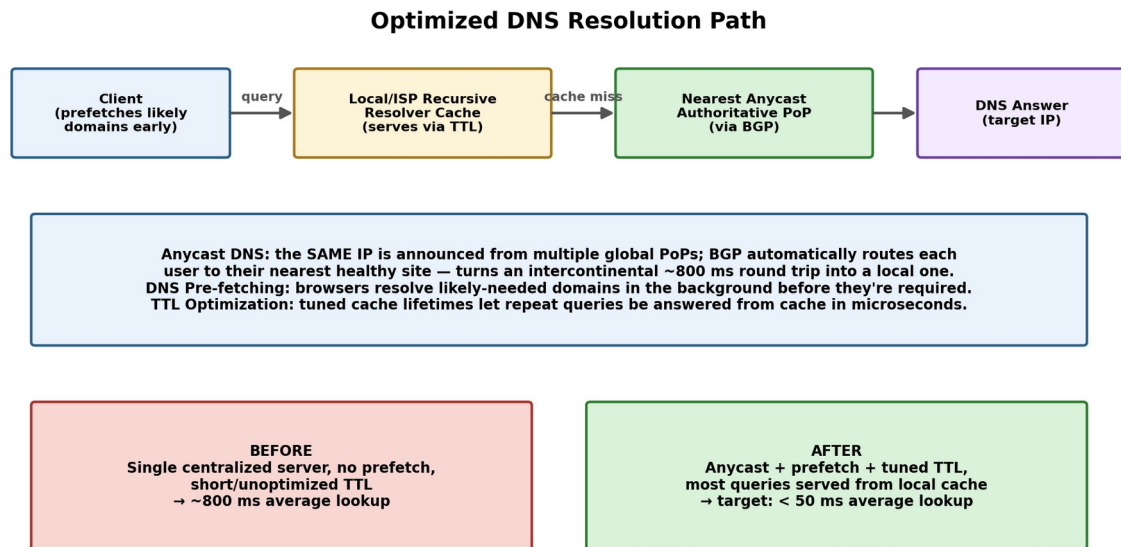


Figure 3 – Proposed low-latency DNS resolution path combining Anycast, pre-fetching and TTL tuning.

- **Anycast DNS:** announce the same authoritative server IP from multiple geographically distributed points-of-presence (PoPs); BGP automatically routes each query to the user's nearest healthy PoP, turning an intercontinental round trip into a local/regional one.
- **DNS pre-fetching:** have client browsers/applications proactively resolve domains they are likely to need (e.g. linked resources) in the background, before the request is actually made, hiding the lookup latency entirely from the user's critical path.
- **TTL optimization:** tune record TTLs to balance freshness against cache efficiency — long enough (minutes to an hour, for typically-stable records) that most repeat queries are answered straight from a local/ISP cache, short enough that IP changes still propagate in reasonable time.
- **Redundant infrastructure:** add replica authoritative servers at each PoP plus a tiered layer of recursive-resolver caching (e.g. via a CDN-integrated DNS service) close to end users.

### 3) Advantages and limitations of the proposed architecture:

Advantages: with most repeat queries answered from local cache (microsecond latency) and even cache-miss queries traveling only to the nearest Anycast PoP instead of across the globe, average lookup times realistically fall to single-digit-to-low-double-digit milliseconds — comfortably under the 50 ms target. Anycast also provides built-in fault tolerance: if one PoP fails, BGP simply stops routing traffic to it and shifts that load to the next-nearest healthy site, with no client-side reconfiguration needed, and the architecture scales naturally since load is spread across many PoPs rather than concentrated on one server — well suited to “millions of users worldwide.”

Limitations: Anycast routing depends on BGP convergence, which is not instantaneous and can occasionally cause brief inconsistency (e.g. “route flapping”) during large-scale network changes; long TTLs, while great for performance, mean a changed record (e.g. after a server migration) takes longer to propagate to all cached copies; pre-fetching consumes a small amount of extra DNS query volume/bandwidth for lookups that may end up unused; and operating multiple Anycast PoPs plus redundant authoritative servers is materially more complex and costly to run and monitor than a single centralized server.

3. A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

### Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

### Answer:

**1) Impact of centralized DNS architecture on performance:** Because every query from every user worldwide must reach the single primary DNS server, users far from that server's physical location pay substantial propagation delay purely from distance, which is consistent with the reported ~800 ms figure. Centralization also creates a scalability bottleneck: query volume that would otherwise be spread across many servers is instead concentrated entirely on one, so during high-traffic periods such as promotional events, server-side queuing/processing delay compounds on top of the propagation delay, producing the reported intermittent access delays. Most seriously, it creates a single point of failure — if that one server becomes overloaded or unreachable, DNS resolution (and therefore effective access to the service) fails for every user globally at once, a severe availability risk for a service serving millions of users worldwide.

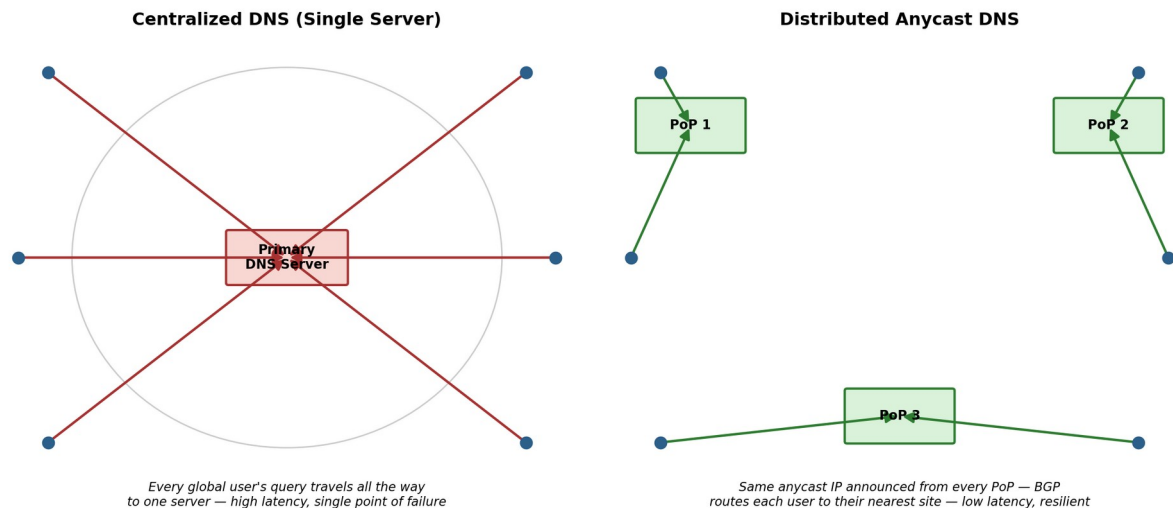


Figure 4 – Centralized single-server DNS vs. a distributed Anycast deployment.

## 2) Distributed DNS solution design:

- Multiple authoritative DNS servers deployed across geographically distributed regions (e.g. Americas, Europe, Asia-Pacific), all serving synchronized replicas of the same zone data via standard zone-transfer mechanisms (AXFR/IXFR).
- Anycast addressing so a single logical IP is announced from every regional site; BGP automatically steers each user's query to their nearest healthy instance — giving both low latency and automatic, network-level failover with no client-side changes needed.
- A tiered caching hierarchy — recursive resolvers at ISPs/CDN edge nodes cache responses locally under TTL control, so only a small fraction of total queries ever need to reach the authoritative layer at all.
- Continuous health-checking/monitoring across all PoPs, automatically withdrawing the Anycast announcement from any unhealthy site so traffic is never routed to a failed instance.

## 3) How TTL, Anycast routing, and DNS caching influence performance and fault tolerance:

- **TTL:** controls the trade-off between freshness and load — short TTLs propagate changes quickly but force frequent re-queries (higher authoritative-server load and more queries paying full resolution latency); long TTLs dramatically cut average latency and server load but slow how quickly changes (e.g. a replaced server IP) reach all resolvers, which can briefly hurt fault tolerance if not paired with active health-checking.
- **Anycast routing:** chiefly a latency and fault-tolerance mechanism — it collapses the effective network distance to each user's nearest PoP (cutting propagation delay) and provides near-automatic failover at the routing layer itself when a PoP goes down, though BGP convergence takes some time (typically seconds) and is not instantaneous.
- **DNS caching:** the single biggest lever for average-case performance, since a cache hit is resolved locally in microseconds with no network round trip at all; it also aids fault tolerance, since resolvers can often continue serving already-cached answers (some even

support explicit “serve-stale” behaviour) even if the authoritative layer is briefly unreachable.

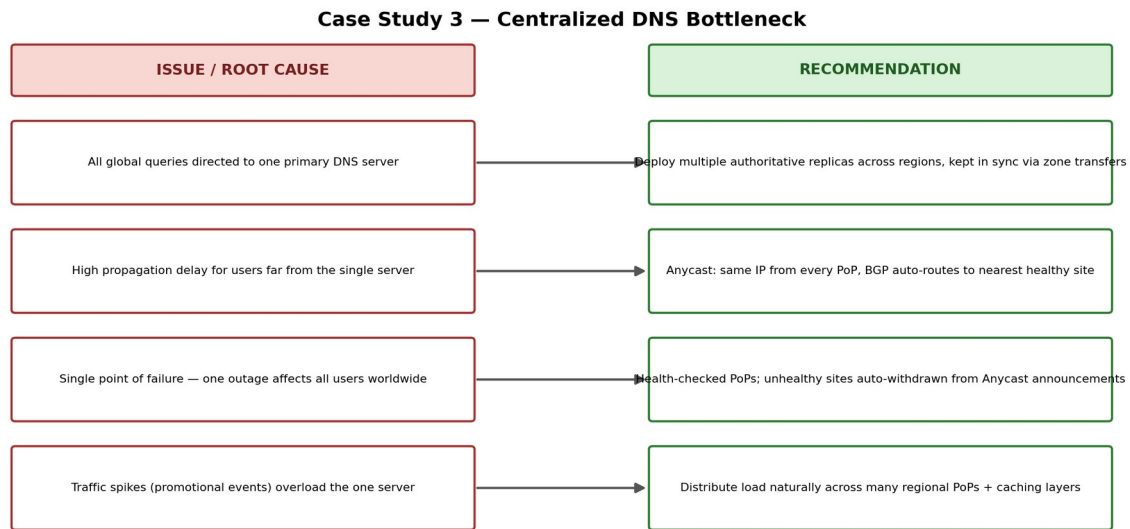


Figure 5 – Summary of Case Study 3 issues mapped to recommended fixes.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

## Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

## Answer:

### 1) Three TCP-related factors behind the latency increase:

- **Flow control:** TCP's receive-window (rwnd) mechanism throttles the sender to match the receiver's currently advertised buffer space. Under the extreme burst of orders at market open, if the receiving application/buffer cannot drain data quickly enough, the advertised window shrinks and forces the sender to pause — even though the network itself still has capacity — directly inflating per-message latency.



- **Nagle's Algorithm:** by default, TCP delays sending small packets, buffering them briefly to coalesce with subsequent small writes into fewer, larger segments (reducing per-packet overhead). For a system sending many small, latency-sensitive messages (individual orders and acknowledgments), this deliberately introduces exactly the kind of delay — further compounded if the receiving side also uses delayed ACK — that can explain a jump from a 1 ms to tens-of-milliseconds baseline.
- **SYN queue limitations:** each new TCP connection begins with a three-way handshake; the listening server maintains a fixed-size SYN backlog for half-open connections awaiting the final ACK. A burst of new/reconnecting trading clients at market open can fill this queue, causing SYN packets to be silently dropped and retried only after a timeout, directly inflating connection-establishment time during precisely the highest-value trading window.

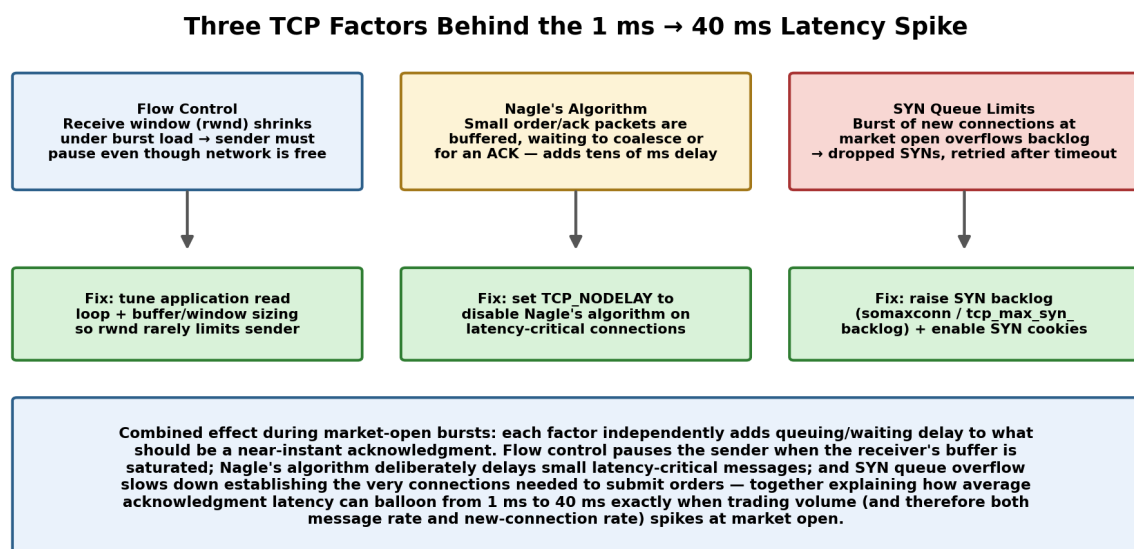


Figure 6 – The three TCP factors behind the market-open latency spike, and their fixes.

## 2) How each factor affects high-frequency transaction processing:

- Flow control throttling adds direct queuing delay to order acknowledgments, since the sender must literally wait before transmitting more data once the window closes — in a system processing thousands of orders per second, even brief window exhaustion multiplies its effect across many in-flight messages.
- Nagle's algorithm (interacting with delayed ACKs) can leave individual small order/ack messages buffered for tens of milliseconds waiting to coalesce or be acknowledged — damaging for a high-frequency trading system whose entire value depends on sub-millisecond-to-low-millisecond responsiveness, since a 40 ms acknowledgment delay could mean an order confirms against a stale price.
- SYN queue overflow lengthens new-session/reconnection time during exactly the highest-traffic, highest-value trading window, when responsiveness matters most and connection churn is likely highest.

### 3) Recommended TCP configuration changes:

- Set the TCP\_NODELAY socket option on trading connections to disable Nagle's algorithm, so small, latency-critical order messages are sent immediately — standard practice for latency-sensitive financial and real-time systems.
- Tune (increase) send/receive socket buffer sizes and window scaling to match the link's bandwidth-delay product, and ensure the application's read loop drains incoming data quickly so the flow-control window rarely becomes the bottleneck.
- Increase the SYN backlog (e.g. Linux's net.core.somaxconn and net.ipv4.tcp\_max\_syn\_backlog) and enable SYN cookies so connection-establishment bursts at market open are absorbed rather than dropped.
- Tune or disable delayed ACK (e.g. TCP\_QUICKACK on Linux) on the receiving side, to avoid it compounding with Nagle's algorithm on the sender.
- For the most latency-critical paths, evaluate a UDP-based or kernel-bypass transport (e.g. DPDK, RDMA) with custom lightweight reliability — the approach many real-world high-frequency trading systems adopt specifically to avoid general-purpose TCP stack overhead altogether, though this is a larger architectural change than a simple configuration tweak.

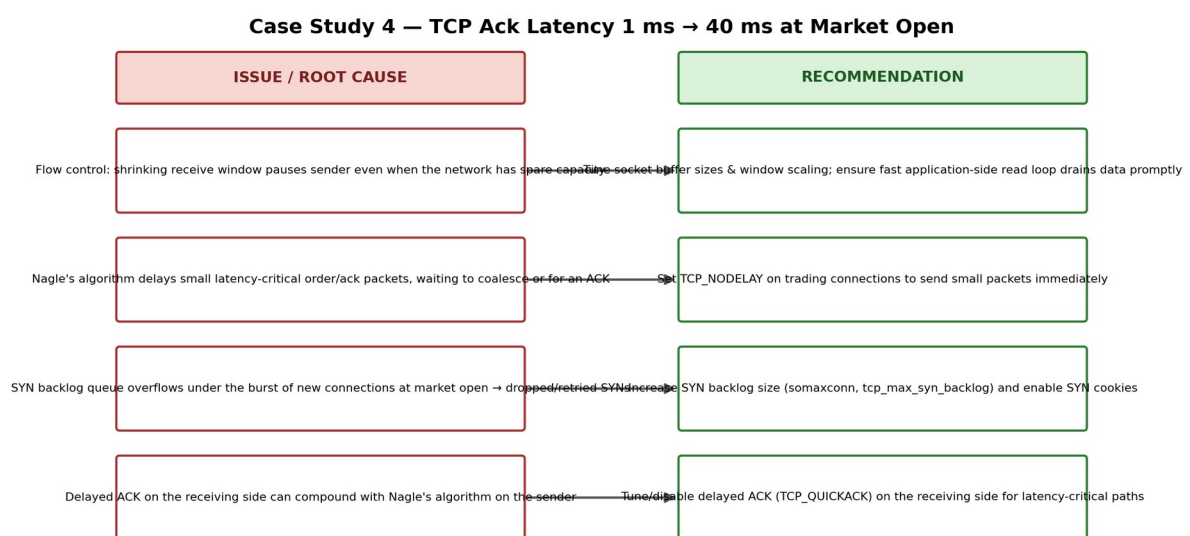


Figure 7 – Summary of Case Study 4 issues mapped to recommended TCP configuration fixes.